ShiftWatch

An ML Reliability and Distribution Shift Monitoring Platform

Project Plan and Technical Blueprint

Project Type:	Full-stack data science / ML monitoring system
Primary Domain:	Urban mobility demand forecasting
Core Methods:	Forecasting, drift detection, anomaly detection, regime modeling
Advanced Layer:	Optimal transport, statistical tests, HMMs, uncertainty quantification
Product Output:	FastAPI backend + interactive dashboard

May 14, 2026

Contents

1 Executive Summary

ShiftWatch is a full-stack machine learning monitoring platform that detects when real-world data begins to behave differently from the data used to train a forecasting model. The platform focuses on urban mobility demand, such as taxi or bike-share trips, where demand patterns can change because of holidays, weather, major events, remote work, public transit disruptions, or long-term neighborhood changes.

The main product idea is simple:

> *A forecasting model may still output predictions even when the world has changed. ShiftWatch monitors whether the current data distribution still resembles the historical baseline, detects anomalies, estimates prediction reliability, and visualizes the changes through an interactive dashboard.*

The project is designed to be attractive for technology recruiters because it combines:

- a clear real-world problem,
- public data,
- end-to-end data engineering,
- machine learning and statistics,
- strong interactive visualizations,
- backend API design,
- business and social impact,
- advanced mathematical ideas underneath the product.

The surface-level product is easy to understand: *monitor demand forecasts and warn when the data changes.* The deeper technical layer is more advanced: *model data as time-indexed probability distributions and detect structural shifts using statistical distances, optimal transport, sequential testing, hidden regimes, and uncertainty-aware forecasting.*

2 Project Identity

2.1 Project Name

ShiftWatch

2.2 Subtitle

> *An ML reliability and distribution shift monitoring platform for urban mobility forecasting.*

Here, **ML** means **machine learning**.

2.3 One-Liner

ShiftWatch detects when urban mobility demand patterns shift away from historical training behavior, helping identify anomalies, hidden regime changes, and periods where forecasting models may become unreliable.

2.4 Recruiter-Friendly Description

I built a full-stack machine learning monitoring dashboard that tracks distribution shift, demand anomalies, and model reliability risks in public urban mobility data using forecasting models, statistical drift metrics, and interactive geospatial visualizations.

2.5 Technical Description

ShiftWatch models incoming data as a time-indexed sequence of probability distributions P_t, compares current windows against a baseline distribution P_0, and combines drift metrics, probabilistic forecasting, anomaly detection, and hidden regime modeling to monitor whether a deployed forecasting model remains reliable.

3 Motivation

Modern machine learning systems are usually trained on historical data. However, real-world systems are not static. User behavior changes, environments change, events happen, and the distribution of incoming data can slowly or suddenly drift away from the training period.

In production machine learning, this creates a serious problem:

> *A model can continue producing predictions even when those predictions are no longer trustworthy.*

For example, an urban mobility forecasting model may be trained on historical taxi demand. During normal periods, the model may work well. But the following events can break normal demand patterns:

- public holidays,
- concerts or sports events,
- extreme weather,
- public transit disruptions,
- changes in commuting behavior,
- airport disruptions,
- neighborhood-level demographic or commercial changes.

A normal forecasting dashboard may only show predicted demand. ShiftWatch goes further by asking:

> *Should we still trust this forecast under the current data environment?*

This is the central motivation of the project.

4 Real-World Examples

4.1 Example 1: Ride-Hailing Demand Forecasting

A ride-hailing platform needs to forecast demand across city zones so it can allocate drivers efficiently. The model was trained on historical demand patterns. However, a major concert causes a sudden demand spike near a stadium. ShiftWatch detects that the current demand distribution differs sharply from baseline behavior and raises a high-drift alert.

4.2 Example 2: Public Transit Disruption

A subway delay causes commuters to switch to taxis or bike-share services. Demand shifts from normal commute corridors into nearby taxi zones. A standard model may underpredict demand because the disruption is unusual. ShiftWatch detects both geospatial drift and residual anomalies.

4.3 Example 3: Long-Term Remote Work Shift

Suppose weekday downtown commute demand gradually decreases over several months, while residential neighborhood demand increases. This is not a one-day anomaly but a structural distribution shift. ShiftWatch can track long-term changes by comparing rolling time windows against the original baseline period.

4.4 Example 4: Model Reliability Risk

The forecasting model may still produce point predictions, but uncertainty increases and prediction residuals become abnormal. ShiftWatch combines drift score, uncertainty score, anomaly score, and recent prediction error into a single model reliability score.

5 Core Problem and Research Question

5.1 Main Problem

Machine learning forecasting models can fail silently when incoming data no longer resembles the training data. The challenge is to detect this change early, explain what changed, and present the information in a form that users can understand.

5.2 Main Question

Can we build an interactive monitoring system that detects distribution shift, demand anomalies, and model reliability risks before forecasting performance degrades severely?

5.3 Subquestions

1. Is the current data distribution significantly different from the baseline training period?
2. Which features or locations are responsible for the shift?
3. Is the shift temporary, seasonal, anomalous, or structural?
4. Is the forecasting model becoming less reliable?
5. Can the shift be explained clearly using visualizations and interpretable metrics?

6 Brief Answer and Platform Concept

The answer is to treat data as an evolving distribution.

At each time window t, the system observes a sample:

$$X_t = \{x_{t,1}, x_{t,2}, \ldots, x_{t,n_t}\}.$$

This sample is interpreted as coming from a distribution:

$$X_t \sim P_t.$$

The historical reference or training period is represented by a baseline distribution:

$$X_0 \sim P_0.$$

If the current distribution P_t is close to P_0, the model environment is considered stable. If the distance

$$d(P_t, P_0)$$

becomes large, the system raises a drift warning.

ShiftWatch then combines this distributional monitoring with forecasting residuals, anomaly detection, and hidden regime detection. The result is a dashboard that answers:

- What is expected to happen?
- What actually happened?

- How unusual is the current behavior?
- Which features changed?
- How reliable is the model right now?

7 Domain Choice: Urban Mobility Demand

The recommended primary domain is **urban mobility demand monitoring**. This domain is strong because it is visual, public-data friendly, and understandable to both technical and non-technical audiences.

7.1 Why Urban Mobility Works Well

Criterion	Why It Works
Real-world problem	Demand forecasting affects transportation planning, logistics, ride-hailing, and city operations.
Public data	Taxi, bike-share, and city mobility datasets are publicly available.
Strong visuals	Maps, heatmaps, timelines, zone-level charts, and anomaly plots are naturally engaging.
ML/statistics	Forecasting, anomaly detection, drift detection, clustering, and uncertainty estimation all fit naturally.
Business impact	Better demand forecasts can improve fleet allocation, pricing, staffing, and operations.
Social impact	Mobility monitoring can support accessibility, congestion analysis, and city planning.
Recruiter appeal	The final result looks like a real analytics product, not just a notebook.

7.2 Suggested Public Data Sources

Potential datasets include:

- NYC Taxi and Limousine Commission trip record data: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
- NYC TLC Trip Record Data on AWS Open Data: https://registry.opendata.aws/nyc-tlc-trip-records-pds/
- Chicago Taxi Trips data portal: https://data.cityofchicago.org/Transportation/Taxi-Trips-2024-/ajtu-isnz
- Citi Bike system data: https://citibikenyc.com/system-data

For the first version, NYC taxi data is likely the best choice because it includes pickup and drop-off times, trip distances, fare information, and zone-level location identifiers. The taxi zone geometry can be used for city-level maps.

8 High-Level System Architecture

The project follows this architecture:

```
Data Source
    -> ETL Pipeline
    -> Feature Engineering
    -> ML / Statistical Models
    -> Prediction and Monitoring Engine
    -> FastAPI Backend
    -> Interactive Dashboard
```

Mapped specifically to ShiftWatch:

```
Public Urban Mobility Data
    -> Raw trip ingestion
    -> Cleaning and aggregation
    -> Zone-time feature engineering
    -> Forecasting, drift detection, anomaly detection, regime modeling
    -> Reliability and alert engine
    -> FastAPI endpoints
    -> Interactive dashboard with maps, timelines, and alerts
```

9 Core Features

9.1 Feature 1: Demand Forecasting

ShiftWatch forecasts future mobility demand by zone and time window.

Example output:

> Predicted taxi demand in Zone 142 from 5 PM to 6 PM: 312 trips. Prediction interval: $[270, 365]$.

Possible model choices:

- seasonal naive baseline,
- linear regression with lag features,
- random forest,

- XGBoost or LightGBM,
- SARIMA or exponential smoothing,
- Bayesian time-series model,
- neural sequence model for advanced extension.

Recommended MVP choice:

> Seasonal baseline + XGBoost or LightGBM + bootstrap prediction intervals.

9.2 Feature 2: Distribution Shift Detection

The platform compares current data windows against a baseline reference window.

Example output:

> Current demand behavior is significantly different from the baseline. Main contributors: pickup zone distribution, hour-of-day distribution, average trip distance, and airport trip frequency.

Possible drift metrics:

- Wasserstein distance,
- Jensen-Shannon divergence,
- Kolmogorov-Smirnov test,
- Population Stability Index,
- Maximum Mean Discrepancy,
- feature-level standardized mean differences.

9.3 Feature 3: Anomaly Detection

The platform detects unusual demand spikes or collapses.

For a forecast $\hat{y}_t$ with uncertainty estimate $\hat{\sigma}_t$, define an anomaly score:

$$A_t = \frac{|y_t - \hat{y}_t|}{\hat{\sigma}_t}.$$

Large values of A_t indicate observations that are surprising relative to the model's expectation.

Example output:

> Zone 234 has unusually high demand: 3.1 standard deviations above expected demand.

9.4 Feature 4: Hidden Regime Detection

ShiftWatch can classify time windows into hidden regimes such as:

- normal weekday commute,
- weekend nightlife,
- low-demand holiday,
- high-demand event,
- airport surge,
- anomalous disruption.

A Hidden Markov Model can be used:

$$Z_t \in \{1, 2, \ldots, K\},$$

where Z_t is the hidden regime at time t. Observed features satisfy:

$$X_t \mid Z_t = k \sim F_k.$$

The dashboard can show a regime probability timeline:

> Normal regime: 12%, high-demand regime: 73%, anomalous regime: 15%.

9.5 Feature 5: Model Reliability Score

The model reliability score summarizes whether the forecasting model should be trusted under current conditions.

A possible score is:

$$R_t = 100 - \left(w_1 D_t + w_2 U_t + w_3 E_t + w_4 A_t\right),$$

where:

- D_t is the drift score,
- U_t is the uncertainty score,
- E_t is recent prediction error,
- A_t is the anomaly score,
- w_1, w_2, w_3, w_4 are weights.

Example output:

> Model reliability: Medium. Reason: strong feature drift in pickup zone distribution and elevated forecast residuals.

9.6 Feature 6: Drift Explanation Panel

When drift is detected, the platform should explain why.

Example output:

1. Pickup zone distribution changed.
2. Nighttime demand increased.
3. Average trip distance increased.
4. Airport trips decreased.

Possible explanation methods:

- feature-level drift metrics,
- feature importance for prediction error,
- SHAP values for forecasting model,
- zone-level drift heatmaps,
- cluster movement in latent space.

10 Deep Mathematical and Statistical Layer

10.1 Distribution Shift as Probability Measure Evolution

At each time window t, data is viewed as a sample from a probability distribution:

$$X_t \sim P_t.$$

In a stable environment:

$$P_t \approx P_0.$$

Under distribution shift:

$$P_t \neq P_0.$$

The monitoring problem becomes estimating a distance or discrepancy:

$$d(P_t, P_0),$$

and deciding whether it is large enough to indicate meaningful change.

10.2 Wasserstein Distance and Optimal Transport

The Wasserstein distance measures how much work is required to transform one distribution into another. In one dimension, the first Wasserstein distance can be written as:

$$W_1(P, Q) = \int_0^1 \left| F_P^{-1}(u) - F_Q^{-1}(u) \right| du,$$

where F_P^{-1} and F_Q^{-1} are quantile functions.

Interpretation:

> How much effort is needed to move the baseline distribution into the current distribution?

This is useful for mobility data because geographic demand may shift from one region to another. Wasserstein distance can capture the geometry of this movement more naturally than simple bin-by-bin comparison.

10.3 KL Divergence and Jensen-Shannon Divergence

KL divergence is:

$$D_{\mathrm{KL}}(P \| Q) = \int p(x) \log \frac{p(x)}{q(x)} dx.$$

It measures information loss when Q is used to approximate P. However, KL divergence can be unstable when distributions have limited overlap. Jensen-Shannon divergence is often more stable:

$$D_{\mathrm{JS}}(P, Q) = \frac{1}{2} D_{\mathrm{KL}}(P \| M) + \frac{1}{2} D_{\mathrm{KL}}(Q \| M),$$

where

$$M = \frac{1}{2}(P + Q).$$

This is useful for categorical and binned features such as hour of day, zone, payment type, or weekday/weekend indicators.

10.4 Maximum Mean Discrepancy

Maximum Mean Discrepancy compares distributions through kernel embeddings:

$$\mathrm{MMD}^2(P, Q) = \mathbb{E}_{X,X' \sim P}[k(X, X')] + \mathbb{E}_{Y,Y' \sim Q}[k(Y, Y')] - 2\mathbb{E}_{X \sim P, Y \sim Q}[k(X, Y)].$$

Interpretation:

> MMD checks whether two samples look different after being mapped into a rich feature space.

MMD is useful for multivariate drift detection.

10.5 Sequential Change Detection

ShiftWatch should not only detect that something changed. It should also estimate when the change began.

A classical CUSUM statistic is:

$$S_t = \max(0, S_{t-1} + x_t - \mu_0 - k).$$

If

$$S_t > h,$$

an alarm is triggered.

Interpretation:

> Small suspicious deviations accumulate over time until there is enough evidence of a real shift.

This is useful for production-style monitoring because it can detect gradual drift before performance collapse.

10.6 Hidden Markov Models

A Hidden Markov Model represents the system as switching between latent regimes.

$$Z_t \sim \text{Markov transition},$$

$$X_t \mid Z_t = k \sim F_k.$$

For urban mobility, the hidden states may correspond to commute behavior, nightlife behavior, holiday behavior, event-driven behavior, or disruption behavior.

The dashboard can display:

- most likely regime over time,
- posterior probability of each regime,
- state transition matrix,
- examples of days or hours belonging to each regime.

10.7 Bayesian or Bootstrap Uncertainty

A forecasting system should not only output a point estimate $\hat{y}_t$. It should estimate uncertainty:

$$p(y_t \mid \mathcal{D}).$$

The dashboard can display prediction intervals:

$$\hat{y}_t \pm 1.96\hat{\sigma}_t.$$

This is important because model reliability is not only about accuracy. It is also about confidence and calibration.

10.8 Embedding Drift

Each time window can be transformed into a latent representation:

$$z_t = \phi(X_t),$$

where ϕ may be PCA, UMAP, an autoencoder, or a vector of summary statistics.

The dashboard can then visualize points z_t in two dimensions. Normal days should cluster together. Holidays, disruptions, and event days should appear far from the normal cluster.

This creates a visually compelling "latent behavior map" of the city.

11 Data Pipeline

11.1 Raw Data

The raw data may contain:

- pickup timestamp,

- drop-off timestamp,
- pickup zone,
- drop-off zone,
- trip distance,
- fare amount,
- passenger count,
- payment type,
- rate code,
- vendor ID.

11.2 ETL Pipeline

The ETL pipeline should:

1. load raw trip data,
2. filter invalid records,
3. parse timestamps,
4. remove impossible or extreme values,
5. aggregate trips by time window and zone,
6. join zone geometry for maps,
7. save processed datasets.

Recommended aggregation levels:

- hourly demand by pickup zone,
- daily demand by pickup zone,
- city-wide hourly demand,
- zone-level summary statistics.

11.3 Feature Engineering

Possible features:

- demand count by zone and hour,
- lagged demand: y_{t-1}, y_{t-24}, y_{t-168},
- rolling averages,
- rolling standard deviations,
- hour of day,
- day of week,
- weekend indicator,
- holiday indicator,
- average trip distance,
- average fare,
- pickup/drop-off imbalance,
- zone-level spatial features,
- airport indicator,
- borough or region indicator.

12 Modeling Modules

12.1 Forecasting Module

Goal:

> Predict future demand y_{t+h} for a zone or city-wide time series.

Suggested progression:

1. seasonal naive model as baseline,
2. linear regression with lag features,
3. XGBoost or LightGBM,
4. uncertainty intervals through bootstrap residuals,
5. optional Bayesian model for advanced extension.

Evaluation metrics:

- MAE,
- RMSE,
- MAPE or sMAPE,
- prediction interval coverage,
- calibration of uncertainty intervals.

12.2 Drift Detection Module

Goal:

> Compare current window P_t with baseline window P_0.

Feature-level drift:

- KS test for continuous features,
- PSI for binned features,
- Jensen-Shannon divergence for categorical distributions,
- Wasserstein distance for continuous or spatial distributions.

Global drift score:

$$D_t = \sum_{j=1}^{p} \alpha_j d_j(P_{t,j}, P_{0,j}),$$

where d_j is the feature-level drift metric for feature j.

12.3 Anomaly Detection Module

Goal:

> Detect time-zone combinations where observed demand is unusually far from predicted demand.

Residual:

$$e_t = y_t - \hat{y}_t.$$

Standardized anomaly score:

$$A_t = \frac{|e_t|}{\hat{\sigma}_t}.$$

Alert rule:

$$A_t > \tau.$$

For example, $\tau = 3$ corresponds to a three-standard-deviation warning rule.

12.4 Regime Detection Module

Goal:

> Identify latent states of city behavior.

Possible approaches:

- K-means clustering on daily profiles,
- Gaussian mixture models,
- Hidden Markov Models,
- Bayesian change point detection as an advanced extension.

12.5 Reliability Engine

The reliability engine combines drift, uncertainty, error, and anomaly scores into a product-friendly health metric.

Possible formula:

$$R_t = \max\left(0, 100 - \left(w_1 D_t + w_2 U_t + w_3 E_t + w_4 A_t\right)\right).$$

Interpretation:

- $R_t \geq 80$: stable,
- $60 \leq R_t < 80$: warning,
- $R_t < 60$: critical.

13 FastAPI Backend

The backend exposes model outputs through clean API endpoints.

Suggested endpoints:

```
GET /health
GET /forecast?zone_id=...&horizon=...
GET /drift?window=...
GET /anomalies?date=...
GET /regimes?start=...&end=...
GET /reliability?zone_id=...
GET /features/drift-contributors?window=...
```

Example response for `/reliability`:

```
{
  "zone_id": 142,
  "timestamp": "2025-03-14T18:00:00",
  "reliability_score": 68.4,
  "status": "warning",
  "main_reasons": [
    "pickup zone distribution drift",
    "elevated forecast residuals",
    "higher-than-usual evening demand"
  ]
}
```

14 Interactive Dashboard

14.1 Page 1: Executive Overview

Purpose:

> Let a viewer understand the system status in ten seconds.

Main cards:

- current status: Stable / Warning / Critical,
- global drift score,
- model reliability score,
- number of active anomaly zones,
- current detected regime,

- most affected feature.

Example:

```
Status: Warning
Global drift score: 0.72
Model reliability: 68 / 100
Current regime: High-demand event
Active anomaly zones: 4
Main contributor: pickup zone distribution
```

14.2 Page 2: Demand Forecasting

Visuals:

- actual vs predicted demand,
- prediction intervals,
- future demand heatmap,
- zone selector,
- model error summary.

14.3 Page 3: Distribution Drift Monitor

Visuals:

- drift score timeline,
- baseline vs current feature distributions,
- feature-level drift ranking,
- Wasserstein distance over time,
- categorical distribution comparison.

14.4 Page 4: Geospatial Drift Map

Visuals:

- zone-level demand heatmap,
- drift severity by zone,
- anomaly markers,
- baseline demand map vs current demand map.

This is one of the most important pages because it makes the project visually impressive.

14.5 Page 5: Anomaly Detection

Visuals:

- anomaly timeline,
- ranked anomaly table,
- observed vs expected demand,
- zone-level anomaly heatmap.

14.6 Page 6: Regime Detection

Visuals:

- HMM state probability timeline,
- colored regime bands,
- transition matrix,
- representative examples of each regime.

14.7 Page 7: Latent Behavior Explorer

Visuals:

- PCA or UMAP projection of time windows,
- points colored by regime,
- points sized by drift score,
- tooltips showing date, demand, regime, and reliability score.

Interpretation:

> Normal weekdays cluster together. Holidays, disruptions, and event days appear as separate regions in latent space.

15 Technology Stack

15.1 Core Stack

Layer	Recommended Tools
Data processing	Python, Pandas, Polars, NumPy
Modeling	scikit-learn, XGBoost, LightGBM, statsmodels
Drift detection	SciPy, POT, custom metrics
Regime modeling	hmmlearn, scikit-learn, custom HMM implementation
Backend	FastAPI, Pydantic, Uvicorn
Dashboard	Streamlit, Plotly, or React + Plotly
Database	SQLite for MVP, PostgreSQL for advanced version
Deployment	Docker, Render, Railway, Fly.io, or Streamlit Cloud
Testing	pytest
Version control	Git and GitHub

15.2 Recommended Dashboard Choice

For speed and portfolio quality, start with **Streamlit + Plotly**. This allows fast iteration and strong visuals. Later, if you want a more software-engineering-heavy project, migrate the dashboard to React.

16 Repository Structure

A clean repository structure:

```
shiftwatch/
|
|-- README.md
|-- requirements.txt
|-- pyproject.toml
|-- docker-compose.yml
|
|-- data/
|   |-- raw/
|   |-- processed/
|   |-- samples/
|
|-- notebooks/
|   |-- 01_eda.ipynb
|   |-- 02_feature_engineering.ipynb
|   |-- 03_forecasting.ipynb
|   |-- 04_drift_analysis.ipynb
|
|-- src/
|   |-- ingestion/
|   |   |-- load_data.py
|   |-- etl/
|   |   |-- preprocess.py
```

```
|   |-- features/
|   |   |-- build_features.py
|   |-- models/
|   |   |-- forecasting.py
|   |   |-- anomaly.py
|   |   |-- drift.py
|   |   |-- regime.py
|   |-- engine/
|   |   |-- monitoring_engine.py
|   |-- utils/
|       |-- config.py
|
|-- api/
|   |-- main.py
|   |-- routes/
|       |-- forecast.py
|       |-- drift.py
|       |-- anomalies.py
|       |-- regimes.py
|       |-- reliability.py
|
|-- dashboard/
|   |-- app.py
|
|-- reports/
|   |-- architecture.md
|   |-- methodology.md
|   |-- results.md
|
|-- tests/
    |-- test_features.py
    |-- test_drift.py
    |-- test_api.py
```

17 Implementation Roadmap

17.1 Phase 0: Project Setup

Deliverables:

- GitHub repository,
- virtual environment,
- project structure,
- README skeleton,

- initial architecture diagram.

17.2 Phase 1: Data Ingestion and Cleaning

Deliverables:

- download or sample public mobility data,
- load raw trip records,
- clean timestamps and invalid values,
- aggregate by zone and time,
- create processed dataset.

17.3 Phase 2: Exploratory Data Analysis

Deliverables:

- demand by hour,
- demand by day of week,
- demand by zone,
- trip distance distribution,
- fare distribution,
- initial maps and heatmaps.

17.4 Phase 3: Forecasting Model

Deliverables:

- baseline seasonal forecasting model,
- XGBoost or LightGBM model,
- train/test split by time,
- evaluation metrics,
- prediction interval approximation.

17.5 Phase 4: Drift Detection

Deliverables:

- baseline reference window,
- rolling current windows,
- feature-level drift scores,
- global drift score,
- drift timeline visualizations.

17.6 Phase 5: Anomaly Detection

Deliverables:

- forecast residuals,
- standardized anomaly scores,
- anomaly thresholds,
- ranked anomaly table,
- anomaly heatmap.

17.7 Phase 6: Reliability Engine

Deliverables:

- reliability score formula,
- stable/warning/critical classification,
- explanation rules,
- summary JSON output.

17.8 Phase 7: FastAPI Backend

Deliverables:

- API routes,
- Pydantic schemas,
- model output serialization,
- local API documentation,
- basic endpoint tests.

17.9 Phase 8: Dashboard MVP

Deliverables:

- executive overview page,
- forecast page,
- drift monitor page,
- anomaly page,
- geospatial heatmap.

17.10 Phase 9: Advanced Modeling

Deliverables:

- HMM regime detection,
- regime timeline,
- latent space visualization,
- transition matrix,
- drift explanation panel.

17.11 Phase 10: Deployment and Polish

Deliverables:

- deployed dashboard,
- Dockerfile,
- polished README,
- methodology report,
- screenshots or GIF demo,
- resume bullets,
- architecture diagram.

18 Minimum Viable Product

The MVP should include:

1. Public mobility data ingestion.
2. Cleaned and aggregated zone-time demand dataset.
3. Demand forecasting model.
4. Drift detection using Wasserstein distance, KS tests, and PSI.
5. Forecast residual anomaly detection.
6. Model reliability score.
7. FastAPI backend.
8. Streamlit or React dashboard.
9. Executive overview page.
10. Forecasting and drift visualization pages.

This version is already resume-worthy.

19 Advanced Extensions

After the MVP, possible extensions include:

- HMM-based hidden regime detection,
- Bayesian uncertainty intervals,
- online change point detection,
- embedding drift monitoring,
- alert simulation system,
- retraining recommendation engine,
- database-backed dashboard,
- Dockerized deployment,
- CI/CD with GitHub Actions,
- React frontend for a more polished product feel.

20 Evaluation Plan

20.1 Forecasting Evaluation

Use time-based validation, not random splits. Metrics:

- MAE,
- RMSE,
- sMAPE,
- prediction interval coverage,
- reliability score behavior during unusual periods.

20.2 Drift Detection Evaluation

Evaluation can include:

- known holiday periods,
- manually identified event periods,
- synthetic drift injection,
- comparing normal weeks vs abnormal weeks,
- feature-level interpretation checks.

20.3 Anomaly Detection Evaluation

Evaluation can include:

- top anomaly inspection,
- correlation with known holidays or events,
- residual distribution analysis,
- false-positive review.

21 Dashboard Design Principles

The dashboard should prioritize clarity and visual impact.

21.1 Principles

- show status first, details second,
- use maps for spatial intuition,
- use timelines for drift and reliability,
- use simple language for alerts,
- explain why the system raised a warning,
- make the advanced statistics visible but not overwhelming.

21.2 Example Alert Copy

```
Warning: Current demand behavior differs from baseline.

Main reasons:
1. Evening demand increased in central zones.
2. Airport trips dropped below expected levels.
3. Forecast residuals are unusually large.

Recommendation:
Review recent demand patterns and consider model retraining if this persists.
```

22 Business and Social Impact

22.1 Business Impact

ShiftWatch can help organizations:

- allocate drivers or vehicles more efficiently,
- detect demand surges earlier,
- reduce forecasting failures,
- identify when models need retraining,
- monitor operational risk,
- improve trust in ML systems.

22.2 Social Impact

For cities and public agencies, a similar system could help:

- monitor mobility disruptions,

- understand changes in commuting behavior,
- identify underserved regions,
- improve transportation planning,
- support congestion analysis.

23 How to Present This on GitHub

23.1 README Structure

A strong README should include:

1. Project title and one-liner.
2. Problem statement.
3. Dashboard screenshots or GIF.
4. Architecture diagram.
5. Data source description.
6. Methodology overview.
7. Key features.
8. How to run locally.
9. API documentation.
10. Results and limitations.
11. Future improvements.

23.2 README Opening Draft

> Modern machine learning systems are often trained under the assumption that future data will look like historical data. In real-world environments, this assumption frequently breaks: demand patterns shift, anomalies emerge, and predictive models silently degrade.
>
> ShiftWatch is an interactive ML reliability monitoring platform that detects distribution shift, demand anomalies, hidden regime changes, and model reliability risks in urban mobility data. It combines public mobility datasets, forecasting models, statistical drift metrics, regime detection, and interactive geospatial dashboards to help users understand when and why a forecasting model may become unreliable.

24 Resume Positioning

24.1 General Resume Bullet

> Built **ShiftWatch**, a full-stack ML monitoring platform that detects distribution shift, demand anomalies, and model reliability risks in public urban mobility data using forecasting models, statistical drift metrics, and interactive geospatial dashboards.

24.2 Technical Resume Bullets

- Designed an end-to-end data pipeline from raw trip records to zone-level forecasting features, drift metrics, anomaly scores, and dashboard-ready model outputs.
- Implemented statistical drift detection using Wasserstein distance, Jensen-Shannon divergence, KS tests, PSI, and feature-level distribution comparisons.
- Developed demand forecasting and residual-based anomaly detection modules with uncertainty-aware reliability scoring.
- Built a FastAPI backend and interactive dashboard for exploring forecasts, drift timelines, anomaly heatmaps, hidden regimes, and model reliability status.
- Modeled urban mobility behavior as a time-indexed sequence of probability distributions and used statistical distance metrics to detect structural changes in demand behavior.

25 Why This Project Is Strategically Strong

ShiftWatch is a strong resume project because it combines product thinking with advanced modeling.

The front-facing product is simple:

> *A dashboard that tells you when demand behavior changes and when your model may be unreliable.*

The back-facing mathematical layer is sophisticated:

> *Time-indexed probability distributions, optimal transport, statistical tests, sequential change detection, hidden regimes, and uncertainty-aware forecasting.*

This combination is important. Recruiters can understand the product quickly, while technical reviewers can appreciate the depth underneath.

26 Potential Risks and Mitigations

Risk	Mitigation
Data size too large	Start with one city, one year, or sampled monthly data. Use Parquet and Polars if needed.
Dashboard too complex	Build MVP pages first: overview, forecast, drift, anomaly map. Add advanced pages later.
Drift metrics hard to interpret	Add feature-level explanations and simple alert copy.
No true live data	Simulate streaming using historical rolling windows.
Model accuracy imperfect	Emphasize monitoring and reliability, not perfect prediction.
Too much math for recruiters	Put math in methodology section; keep dashboard product-focused.

27 Final Recommended Build Order

The best order is:

1. Data ingestion and cleaning.
2. EDA and first visuals.
3. Aggregated demand dataset.
4. Baseline forecast model.
5. XGBoost or LightGBM forecast model.
6. Forecast residual anomaly score.
7. Drift detection metrics.
8. Reliability score.
9. FastAPI endpoints.
10. Streamlit dashboard MVP.
11. Geospatial maps.
12. HMM regime detection.
13. Latent space explorer.
14. Deployment and README polish.

28 Conclusion

ShiftWatch is a strong second flagship project after a trading algorithm project because it moves into a different but highly relevant area: production machine learning reliability. It demonstrates that the builder can work beyond pure modeling by designing data pipelines, APIs, dashboards, monitoring logic, and user-facing analytics.

The project has the exact profile desired for a resume-oriented data science or tech project:

- clear real-world problem,
- public data,
- strong visuals,
- machine learning and statistics,
- business and social impact,
- full-stack system architecture,
- advanced mathematical depth underneath.

The final message of the project is:

> *Forecasting is not enough. Real-world ML systems also need to know when the world has changed. ShiftWatch monitors that change.*